第2章 数据类型与表达式

2.1 学习目标

- 理解常量与变量的概念
- 掌握C语言的基本数据类型
- 理解字符编码（ASCII）与字符类型，会用转义字符
- 熟练使用各类运算符：算术、自增自减、赋值与复合赋值、逗号、关系、逻辑、位运算
- 掌握字符的输入与输出（`printf` / `scanf` / `putchar` / `getchar`）
- 理解类型转换规则

2.2 标识符与关键字

标识符命名规则

- 由字母、数字、下划线组成
- 第一个字符必须是字母或下划线
- 区分大小写（`sum` 和 `Sum` 是不同的）
- 不能使用关键字

```
// ? 合法的标识符
int age;
int _count;
int student2;
int MAX_VALUE;

// ? 非法的标识符
int 2student;    // 不能以数字开头
int my-age;      // 不能使用连字符
int float;       // 不能使用关键字
```

32个关键字（C89）

```
auto    break   case    char    const   continue
default do      double  else    enum    extern
float   for     goto    if      int     long
register return  short  signed  sizeof  static
struct  switch  typedef union   unsigned void
volatile while
```

2.3 常量

2.3.1 整型常量

```
123      // 十进制
0123     // 八进制（以0开头）
0x1A     // 十六进制（以0x开头）
```

2.3.2 实型常量

```
3.14     // 小数形式
1.2e5    // 指数形式 = 1.2 × 10^5
-3.0E-2  // 指数形式 = -3.0 × 10^(-2)
```

2.3.3 字符常量

字符常量是用**单引号**括起来的**一个**字符：

```
'A'      // 字母
'1'      // 字符'1'，它的编码是 49，不是数字 1
'0'      // 字符'0'，它的编码是 48，不是数字 0
' '      // 空格（注意单引号中间有一个空格）

'AB'     // ? 错误：单引号内只能有一个字符
" A "    // ? 这是字符串，不是字符常量
```

> 字符常量的本质是它对应的**编码值（整数）**，可以像整数一样参与运算。
> 具体见 **2.5 字符编码与字符类型**。

2.3.4 转义字符

有些字符没法直接写出来（比如换行），有些字符会与 C 语言的语法符号冲突（比如引号、反斜杠），这时就要用**转义字符**：以反斜杠 `\` 开头的特殊字符序列，"转义"就是"改变它原来的含义"。

转义字符	含义	ASCII 编码
`\n`	换行（newline），光标移到下一行行首	10
`\t`	水平制表符（tab），跳到下一个制表位	9
`\r`	回车，光标回到本行行首	13
`\0`	空字符（字符串结束标志）	0
`\\`	反斜杠 `\`	92
`\'`	单引号 `'`	39
`\"`	双引号 `"`	34

转义字符	含义	ASCII 编码
`\a`	响铃/蜂鸣	7
`\b`	退格（删除前一个字符）	8
`\f`	换页	12
`\v`	垂直制表	11
`\ddd`	1~3 位**八进制**数表示的字符，如 `\101`	65（即 `'A'`）
`\xhh`	1~2 位**十六进制**数表示的字符，如 `\x41`	65（即 `'A'`）

使用要点：

- `'\n'` 是**一个**字符（换行符），`"\n"` 是**一个字符串**（换行符 + 结束标志）
- 表示反斜杠必须写成 `'\\'`，写成 `'\'` 会编译错误（`\` 把后面的 `'` 转义了）
- `'\0'` 与 `'0'` 完全不同：前者编码是 0（空字符），后者编码是 48（字符零）
- `'\101'`、`'\x41'`、`'A'` 三个写法是**同一个字符**
- 转义字符在 `printf` 的格式串里同样有效，写多行输出时非常常用

案例 2-1 转义字符大集合

```c
#include <stdio.h>
int main()
{
    printf("1. 换行符: 第一行\n第二行\n");
    printf("2. 引号: 他说: \"你好\"\n");
    printf("3. 反斜杠: C:\\Dev-Cpp\\demo.c\n");
    printf("4. 单引号: 字符常量 'A'\n");
    printf("5. 八进制\\101 = %c，十六进制\\x41 = %c\n", '\101', '\x41');
    printf("6. 响铃: \a\n");
    printf("7. 退格: ABC\bD\n");      // 退格删掉 C，输出 ABD
    return 0;
}
```

案例 2-2 用 `\t` 打印对齐的表格

```c
#include <stdio.h>
int main()
{
    printf("姓名\t语文\t数学\t总分\n");
    printf("小明\t95\t88\t183\n");
    printf("小红\t78\t96\t174\n");
    printf("小刚\t100\t100\t200\n");
    return 0;
}
```

> 说明：`\t` 会跳到下一个制表位（每 8 列一个），所以短名字和长名字都能大致对齐。如果想精确控制列宽，用 `printf("%-8s", "小明")` 更方便。

2.3.5 符号常量（#define）

```c
#include <stdio.h>
#define PI 3.14159       // 定义符号常量
#define MAX 100
const float PI = 3.14159;  //const关键字是C99标准中新引入的，在程序中定义常量
int main()
{
    float r = 5.0;
    float area = PI * r * r;
    printf("面积 = %.2f\n", area);
    printf("MAX = %d\n", MAX);
    return 0;
}
```

2.4 变量

2.4.1 变量的定义与初始化

```c
int a;            // 定义（未初始化，值为随机数）
int b = 10;       // 定义并初始化
float x, y, z;    // 同时定义多个同类型变量
char ch = 'A';    // 字符变量
```

2.4.2 基本数据类型

类型	关键字	占用字节	取值范围	格式符
整型	`int`	4	-2^31 ~ 2^31-1	`%d`
短整型	`short`	2	-32768 ~ 32767	`%hd`
长整型	`long`	4/8	更大范围	`%ld`
字符型	`char`	1	-128 ~ 127	`%c`
单精度浮点	`float`	4	±3.4e-38~±3.4e38	`%f`
双精度浮点	`double`	8	±1.7e-308~±1.7e308	`%lf`
空型	`void`	—	表示无类型	—

2.4.3 用 sizeof 查看类型大小

```c
#include <stdio.h>
int main()
{
    printf("int:    %d 字节\n", sizeof(int));
    printf("char:   %d 字节\n", sizeof(char));
    printf("float:  %d 字节\n", sizeof(float));
    printf("double: %d 字节\n", sizeof(double));
    printf("short:  %d 字节\n", sizeof(short));
    printf("long:   %d 字节\n", sizeof(long));
    return 0;
}
```

2.5 字符编码与字符类型

2.5.1 什么是字符编码

计算机内部只会存 0 和 1，要让计算机表示 `A`、`9`、`+` 这些字符，就必须给每个字符规定一个**编号**，这套编号规则就叫**字符编码**。C 语言中最常用的是 **ASCII 码**（美国信息交换标准代码），一共定义了 128 个字符，用一个字节就能装下。

ASCII 码表（常用部分，十进制）：

十进制	字符	说明
0	`\0`	空字符（字符串结束标志）
9	`\t`	制表符
10	`\n`	换行符
32	`' '`	空格
48 ~ 57	`'0'` ~ `'9'`	数字字符
65 ~ 90	`'A'` ~ `'Z'`	大写字母
97 ~ 122	`'a'` ~ `'z'`	小写字母

三条必须记住的规律：

1. `'0'` = **48**（不是 0！），`'A'` = **65**，`'a'` = **97**
2. 同类字符的编码是**连续**的，所以 `'B'` = `'A' + 1`，`'b'` = `'a' + 1`
3. 同一个字母的大写和小写**相差 32**，即 `'a' - 'A' = 32`

2.5.2 字符类型 char

`char` 是 C 语言中**最小的整型类型**，占 1 个字节，专门用来存放字符——存的其实就是字符的编码值。

```
char ch = 'A';    // 变量 ch 的内存里实际存放的是 65
printf("%c\n", ch);   // 以字符形式输出：A
printf("%d\n", ch);   // 以整数形式输出：65
```

类型	占用字节	取值范围	格式符
`char`（多数编译器等同 `signed char`）	1	-128 ~ 127	`%c` 输出字符，`%d` 输出编码
`unsigned char`	1	0 ~ 255	`%c`

> ASCII 码只用到 0~127，所以 `char` 是有符号还是无符号都能装下，平时不用纠结；但如果要存 128~255 的编码（如扩展 ASCII 或中文的某个字节），就必须用 `unsigned char`。

字符常量 vs 字符串：

```
char ch = 'A';     // 字符常量：单引号，1 个字符，占 1 字节，值为 65
char s[] = "A";    // 字符串：双引号，内容为 'A' + '\0'，占 2 字节
```

2.5.3 字符与整数

既然 `char` 里存的是编码（整数），字符就能像整数一样参与运算，也能在 `%c` 与 `%d` 之间随意切换：

写法	结果	说明
`printf("%c", 65)`	`A`	整数按字符输出
`printf("%d", 'A')`	`65`	字符按整数输出
`'9' - '0'`	`9`	字符数字 → 数值数字
`9 + '0'`	`'9'`	数值数字 → 字符数字
`'A' + 32`	`'a'`	大写转小写
`'a' - 32`	`'A'`	小写转大写
`ch ^ 32`	大小写互换	用异或翻转编码的第 5 位（见 2.6.5 位运算符）
`'z' - 'a' + 1`	`26`	字母表共 26 个字母

判断字符类型的常用写法：

```
if (ch >= 'a' && ch <= 'z')   // 是否小写字母
if (ch >= 'A' && ch <= 'Z')   // 是否大写字母
if (ch >= '0' && ch <= '9')   // 是否数字字符
```

案例 2-3 字符编码探秘

```c
#include <stdio.h>
int main()
{
    char ch;
    printf("请输入一个字符: ");
    scanf("%c", &ch);

    printf("字符为 %c，编码为 %d\n", ch, ch);
    printf("前一个字符: %c (%d)\n", ch - 1, ch - 1);
    printf("后一个字符: %c (%d)\n", ch + 1, ch + 1);

    if (ch >= 'A' && ch <= 'Z')
        printf("%c 是大写字母，转成小写是 %c\n", ch, ch + 32);
    else if (ch >= 'a' && ch <= 'z')
        printf("%c 是小写字母，转成大写是 %c\n", ch, ch - 32);
    else if (ch >= '0' && ch <= '9')
        printf("%c 是数字字符，它的数值是 %d\n", ch, ch - '0');
    else
        printf("%c 是其他字符\n", ch);

    return 0;
}
```

运行示例（输入 `G`）：

```
字符为 G，编码为 71
前一个字符: F (70)
后一个字符: H (72)
G 是大写字母，转成小写是 g
```

2.6 运算符与表达式

2.6.1 算术运算符

双目运算符	含义	示例	结果
`+`	加	`5 + 3`	`8`
`-`	减	`5 - 3`	`2`
`*`	乘	`5 * 3`	`15`
`/`	除	`5 / 3`	`1`（整数除法）
`%`	取余（模）	`5 % 3`	`2`

> **注意**：整数除法会截断小数部分。`5 / 3` 结果是 `1` 而不是 `1.6666`。
> 口诀：% 就是问“除完之后还剩多少？”只能用于整数，不能用于小数
> 如果要对小数取余，得用 fmod() 函数（需要 #include <math.h>）。

```c
#include <stdio.h>
int main()
{
    int a = 10, b = 3;
    printf("a + b = %d\n", a + b);   // 13
    printf("a - b = %d\n", a - b);   // 7
    printf("a * b = %d\n", a * b);   // 30
    printf("a / b = %d\n", a / b);   // 3 (整数除法)
    printf("a %% b = %d\n", a % b);  // 1

    // 浮点数除法
    printf("10.0 / 3 = %f\n", 10.0 / 3);  // 3.333333
    return 0;
}
```

2.6.2 自增自减运算符

把 i 想象成一个计数器，++ 就是“按一下加 1”。
i++（后缀）：你先看一眼当前数字记下来，然后再按一下让数字加 1。
你记下的是按之前的数。
++i（前缀）：你先按一下让数字加 1，然后再看当前数字记下来。
你记下的是按之后的数。

```c
int i = 5;
int a, b;

a = i++;   // 后缀：先赋值，后自增。a=5, i=6
b = ++i;   // 前缀：先自增，后赋值。b=7, i=7
```

```c
#include <stdio.h>
int main()
{
    int x = 5;
    printf("x++ = %d\n", x++);   // 输出5，x变为6
    printf("现在 x = %d\n", x);   // 输出6

    int y = 5;
    printf("++y = %d\n", ++y);   // 输出6，y变为6
    printf("现在 y = %d\n", y);   // 输出6
    return 0;
}
```

2.6.3 赋值运算符与复合赋值运算符

（1）简单赋值

```c
a = 10;       // 把 10 赋给 a
a = b = c = 0;   // 赋值运算符右结合，等价于 a = (b = (c = 0))
```

（2）复合赋值运算符：把"运算"和"赋值"合并成一个运算符，写起来更简洁，也是 C 语言的特色。

复合运算符	等价写法	说明
`+=`	`a = a + b`	加后赋值
`-=`	`a = a - b`	减后赋值
`*=`	`a = a * b`	乘后赋值
`/=`	`a = a / b`	除后赋值
`%=`	`a = a % b`	取余后赋值
`<<=`	`a = a << b`	左移后赋值
`>>=`	`a = a >> b`	右移后赋值
`&=`	`a = a & b`	按位与后赋值
`|=`	`a = a | b`	按位或后赋值
`^=`	`a = a ^ b`	按位异或后赋值

使用要点：

1. **右侧是一个整体。** `a *= b + 3` 等价于 `a = a * (b + 3)`，**不是** `a = a * b + 3`。
2. 复合赋值会**自动把结果转回左边变量的类型。** `int i = 5; i += 2.5;` 结果是 `7`
   C语言把浮点数转成整数时，是直接砍掉小数点后面的部分。
3. 复合赋值只对左边的变量**求值一次**（`a[i++] += 2` 这类写法里自增只执行一次）。
4. 别把 `=` 写成 `==`：`=` 是赋值，`==` 是判断相等。

案例 2-4 复合赋值运算符演示

```c
#include <stdio.h>
int main()
{
    int a = 10;

    a += 5;     printf("a += 5   → %d\n", a);   // 15
    a -= 3;     printf("a -= 3   → %d\n", a);   // 12
    a *= 2;     printf("a *= 2   → %d\n", a);   // 24
    a /= 4;     printf("a /= 4   → %d\n", a);   // 6
    a %= 4;     printf("a %%= 4   → %d\n", a);  // 2

    a <<= 3;    printf("a <<= 3  → %d\n", a);   // 16
    a &= 6;     printf("a &= 6   → %d\n", a);   // 0

    int b = 2;
    b *= 3 + 4;    printf("b *= 3+4 → %d\n", b);   // 14，即 2*(3+4)

    char ch = 'A';
    ch += 1;       printf("ch += 1  → %c\n", ch);  // B
    return 0;
```

```
}
```

2.6.4 逗号运算符

逗号在 C 语言里有两种身份：

1. **分隔符**：`int a, b, c;` 里的逗号、函数参数 `printf("%d", x)` 里的逗号，只是起分隔作用。
2. **逗号运算符**：用逗号把多个表达式连起来，构成**逗号表达式。**

语法：

```
表达式1, 表达式2, 表达式3, ...
```

求值规则：

- 从左到右**依次计算**每个表达式
- 整个逗号表达式的值 = **最后一个表达式的值**
- 逗号运算符的优先级是 C 语言里**最低**的（比赋值 `=` 还低），所以要把整个表达式的值赋给变量时，通常要加括号

```
int x = (1, 2, 3);          // 依次算 1、2、3，x = 3
int y = (a = 5, b = 8);     // 先 a=5，再 b=8，y = 8
```

案例 2-5 逗号运算符

```
#include <stdio.h>
int main()
{
    int a, b, x;

    x = (a = 3, b = a + 4, a + b);   // ① a=3  ② b=7  ③ 值 = a+b = 10
    printf("a = %d, b = %d, x = %d\n", a, b, x);   // 3, 7, 10

    int n = 1;
    n = (n + 2, n * 3);              // 先算 n+2（结果丢掉），再算 n*3
    printf("n = %d\n", n);           // 9

    printf("%d\n", (1, 2, 3));       // 3
    printf("%d\n", (x = 100, x + 1)); // 101，同时 x 被改成 100
    return 0;
}
```

> **常见错误**：`int x = 1, 2, 3;` 这里的逗号是分隔符，会被理解成"定义变量 x=1、2、3"，直接编译报错。

2.6.5 位运算符

位运算符直接对整数的**二进制位**进行操作，**只能用于整型数据**（`int`、`char`、`long`、`unsigned` 等），不能用于 `float`、`double`。

运算符	名称	运算规则
`&`	按位与	两位都为 1 结果才为 1，否则为 0
`|`	按位或	两位只要有一个为 1 结果就为 1
`^`	按位异或	两位**相同为 0，不同为 1**
`~`	按位取反（单目）	1 变 0，0 变 1
`<<`	左移	整体左移，右边空出的位补 0
`>>`	右移	整体右移，正数左边补 0

以 8 位为例（`5` = `0000 0101`，`3` = `0000 0011`）：

表达式	二进制运算过程	结果
`5 & 3`	`0000 0101 & 0000 0011` → `0000 0001`	`1`
`5 | 3`	`0000 0101 | 0000 0011` → `0000 0111`	`7`
`5 ^ 3`	`0000 0101 ^ 0000 0011` → `0000 0110`	`6`
`~5`	`~0000 0101` → `1111 1010`（补码表示）	`-6`
`5 << 1`	`0000 0101` 左移一位 → `0000 1010`	`10`
`5 >> 1`	`0000 0101` 右移一位 → `0000 0010`	`2`

> **易混点**：`&` 与 `&&`、`|` 与 `||` 是完全不同的运算符！
> `5 & 3` 是**按位与**，结果是 `1`；`5 && 3` 是**逻辑与**（都非 0，都为真），结果也是 `1`——巧合而已。
> 看 `4 & 2` = `0`，而 `4 && 2` = `1`，差别立刻显现。

常用技巧（设 `x` 是整数，`n` 是位数）：

目的	写法	说明
判断奇偶	`x & 1`	结果为 1 是奇数，0 是偶数
把第 n 位置 1	`x | (1 << n)`	第 n 位变成 1
把第 n 位清 0	`x & ~(1 << n)`	第 n 位变成 0
把第 n 位翻转	`x ^ (1 << n)`	1 变 0、0 变 1
乘 2 的 n 次方	`x << n`	`x << 1` 等于 x*2*，`x << 3` 等于 *x*8
除以 2 的 n 次方	`x >> n`	`x >> 1` 等于 x/2（向下取整）

> **注意**：
>
> - 负数在内存中以**补码**存放，所以对负数做位运算要先换算成补码，结果看起来会"别扭"（如 `~5 = -6`）。

> - 左移时高位可能移出丢失，导致结果"溢出"。
> - 移位位数不能是负数，一般也不应大于等于该类型的位数。

案例 2-6 位运算基础演示

```c
#include <stdio.h>
int main()
{
    int a = 5, b = 3;                // 5 = 0101, 3 = 0011

    printf("a = %d, b = %d\n", a, b);
    printf("a & b  = %d\n", a & b);   // 1
    printf("a | b  = %d\n", a | b);   // 7
    printf("a ^ b  = %d\n", a ^ b);   // 6
    printf("~a     = %d\n", ~a);      // -6
    printf("a << 1 = %d\n", a << 1);  // 10  (5 * 2)
    printf("a >> 1 = %d\n", a >> 1);  // 2   (5 / 2)

    printf("a && b = %d\n", a && b);  // 1  ← 逻辑与，注意区别
    return 0;
}
```

案例 2-7 用异或交换两个数（不用临时变量）

```c
#include <stdio.h>
int main()
{
    int a = 10, b = 20;
    printf("交换前: a = %d, b = %d\n", a, b);

    a = a ^ b;    // a 变成 a^b
    b = a ^ b;    // b = (a^b)^b = a，b 得到原来的 a
    a = a ^ b;    // a = (a^b)^a = b，a 得到原来的 b

    printf("交换后: a = %d, b = %d\n", a, b);
    return 0;
}
```

> 用到的异或性质：`x ^ x = 0`、`x ^ 0 = x`，且异或满足交换律和结合律。

案例 2-8 输出一个整数的二进制位

```c
#include <stdio.h>
int main()
{
    int n, i;
    printf("请输入一个非负整数: ");
    scanf("%d", &n);

    printf("%d 的二进制为: ", n);
    for (i = 7; i >= 0; i--)          // 从最高位开始，打印低 8 位
    {
```

```
        printf("%d", (n >> i) & 1);   // 先把第 i 位移到最低位，再 & 1 取出
    }
    printf("\n");
    return 0;
}
```

运行示例：输入 `5` → 输出 `00000101`。

> 说明：`n >> i` 把第 i 位移到最低位，`& 1` 把最低位取出来。要看更多位，把 7 改成 15（16 位）或 31（32 位）即可。（`for` 循环详见第 3 章。）

2.6.6 关系运算符

运算符	含义	示例
`>`	大于	`a > b`
`<`	小于	`a < b`
`>=`	大于等于	`a >= b`
`<=`	小于等于	`a <= b`
`==`	等于	`a == b`
`!=`	不等于	`a != b`

> 关系表达式的结果：真为`1`，假为`0`。

```
int a = 5, b = 3;
printf("%d\n", a > b);   // 输出 1（真）
printf("%d\n", a == b);  // 输出 0（假）
printf("%d\n", a != b);  // 输出 1（真）
```

2.6.7 逻辑运算符

运算符	含义	示例	说明
`&&`	逻辑与	`a && b`	两边都为真才为真
`||`	逻辑或	`a || b`	任意一边为真即为真
`!`	逻辑非	`!a`	取反

```c
#include <stdio.h>
int main()
{
    int a = 1, b = 0;   // 1为真, 0为假
    printf("a && b = %d\n", a && b);   // 0
    printf("a || b = %d\n", a || b);   // 1
    printf("!a = %d\n", !a);           // 0
    printf("!b = %d\n", !b);           // 1
    return 0;
}
```

短路求值：

```c
// 示例：短路求值
int x = 0;
int y = (x != 0) && (10 / x > 1);  // 左边为假，右边不执行，不会除零错误
```

2.7 运算符优先级（由高到低）

优先级	运算符	结合方向
1	`()` `[]` `->` `.`	左→右
2	`!` `~` `++` `--` `+`（正） `-`（负） `*`（指针） `&`（取址） `sizeof`	右→左
3	`*` `/` `%`	左→右
4	`+` `-`	左→右
5	`<<` `>>`	左→右
6	`<` `<=` `>` `>=`	左→右
7	`==` `!=`	左→右
8	`&`	左→右
9	`^`	左→右
10	`|`	左→右
11	`&&`	左→右
12	`||`	左→右
13	`?:`（条件运算符）	右→左
14	`=` `+=` `-=` `*=` `/=` `%=` `<<=` `>>=` `&=` `^=` `|=`	右→左
15	`,`（逗号运算符）	左→右

> **口诀**：单算移关与，异或逻条赋 —— 单目 > 算术 > 移位 > 关系 > 位运算 > 逻辑 > 条件 > 赋值
> 记住：**逗号运算符优先级最低**，**赋值运算符次低**，**位运算符比关系运算符低**（这是最容易出错的地方）。

看几个典型优先级例子：

```
int a = 1, b = 2, c = 3;

printf("%d\n", a + b * c);        // 7，先 * 后 +
printf("%d\n", a & 3 == 3);       // 先算 3==3 得 1，再 1 & 1 = 1（!= 按位与优先）
printf("%d\n", (a & 3) == 3);     // 0，加了括号才是"先按位与再比较"
printf("%d\n", a ? b : c);        // 2，条件运算符
printf("%d\n", (a = 5, a + 1));   // 6，逗号表达式整体为 6
// 注意：printf("%d\n", a = 5, a + 1) 里的逗号是"参数分隔符"，不是逗号运算符
```

2.8 类型转换

2.8.1 自动类型转换（隐式转换）

规则：**低精度 → 高精度** 自动转换。

```
char → short → int → long → float → double
```

```
int a = 5;
double b = 3.2;
double result = a + b;    // a自动转为double，结果是8.2
```

2.8.2 强制类型转换（显式转换）

C语言规则：

1. 只要有一个是浮点数，另一个会自动升级成浮点数，然后做浮点除法。
2. 如果两个都是整数，则做整数除法，小数部分直接丢掉。

```
(目标类型) 表达式
```

```
int a = 5, b = 2;
double result1 = a / b;           // 整数除法，result1 = 2.0
double result2 = (double)a / b;   // 强制转换，result2 = 2.5

float x = 3.14;
int y = (int)x;                   // y = 3（截断小数）
```

2.9 字符的输入与输出

2.9.1 输出一个字符

方式	用法	说明
`printf`	`printf("%c", ch);`	最常用，可和其他内容混合输出
`putchar`	`putchar(ch);`	专用于输出单个字符（参数可写字符变量、字符常量或编码值）

```
char ch = 'A';
printf("%c\n", ch);    // 输出 A 并换行
putchar(ch);           // 输出 A，不换行
putchar('\n');         // 输出一个换行
putchar(65);           // 输出 A（65 就是 'A' 的编码）
```

2.9.2 输入一个字符

方式	用法	说明
`scanf`	`scanf("%c", &ch);`	从输入缓冲区读一个字符，`%c` 不跳过空白字符
`getchar`	`ch = getchar();`	从输入缓冲区读一个字符并返回它的编码

```
char ch;
printf("请输入一个字符: ");
ch = getchar();           // 等价于 scanf("%c", &ch);
printf("你输入的是 %c，编码 %d\n", ch, ch);
```

`getchar()` 的返回值是 `int`（为了能返回 `EOF`，值为 -1，表示输入结束），所以更规范、更安全的写法是用 `int` 接收：

```
int ch;
ch = getchar();
if (ch != EOF)
    printf("读到字符: %c\n", ch);
```

> `EOF` 是定义在 `<stdio.h>` 中的符号常量（值 -1），表示"输入结束"。键盘输入时，Windows 下按 `Ctrl+Z` 再回车即可产生 EOF。

2.9.3 输入缓冲区与"吃掉回车"★重点

`scanf` 和 `getchar` 都不是直接向键盘取数据，而是从**输入缓冲区**里取。当你在键盘上输入 `18` 然后按回车时，缓冲区里实际上是 `18\n`：`scanf("%d", &age)` 只取走了 `18`，**把** `\n` **留在了缓冲区**。下一次读字符时就会先读到这个 `\n`，程序看起来像"跳过了输入"。

```
#include <stdio.h>
int main()
```

```c
{
    int age;
    char ch;

    printf("请输入年龄: ");
    scanf("%d", &age);       // 输入 18 后回车，'\n' 残留在缓冲区

    printf("请输入一个字符: ");
    scanf("%c", &ch);        // 读到的其实是 '\n'，这次输入被"跳过"了

    printf("age = %d, ch = [%c]\n", age, ch);
    return 0;
}
```

三种解决办法：

```c
// 方法 1：先用 getchar() 把残留的 '\n' 读走
scanf("%d", &age);
getchar();
scanf("%c", &ch);

// 方法 2（推荐）：在 %c 前加一个空格，表示"跳过所有空白字符"
scanf("%d", &age);
scanf(" %c", &ch);

// 方法 3：把本行剩余内容全部读掉
scanf("%d", &age);
while (getchar() != '\n');
```

> **记住**：`scanf` 中的 `%d`、`%f`、`%s` 会自动跳过前面的空白字符（空格、回车、Tab），而 `%c` **不会**。所以用 `%c` 读字符时，写成 `scanf(" %c", &ch);` 最省事。

2.9.4 字符输入输出案例

案例 2-9 大小写转换器

```c
#include <stdio.h>
int main()
{
    char ch;
    printf("请输入一个字母: ");
    scanf(" %c", &ch);          // 注意 %c 前面有一个空格

    if (ch >= 'A' && ch <= 'Z')
        printf("%c → %c\n", ch, ch + 32);
    else if (ch >= 'a' && ch <= 'z')
        printf("%c → %c\n", ch, ch - 32);
    else
        printf("%c 不是字母\n", ch);

    return 0;
```

```
}
```

案例 2-10 统计一行字符中的字母、数字和其他字符

```c
#include <stdio.h>
int main()
{
    int ch;                              // 用 int 接收才能与 EOF 比较
    int letters = 0, digits = 0, others = 0;

    printf("请输入一行字符，按回车结束: ");
    while ((ch = getchar()) != '\n' && ch != EOF)
    {
        if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z'))
            letters++;
        else if (ch >= '0' && ch <= '9')
            digits++;
        else
            others++;
    }

    printf("字母: %d 个\n", letters);
    printf("数字: %d 个\n", digits);
    printf("其他: %d 个\n", others);
    return 0;
}
```

运行示例：输入 `Abc 123!` → 字母 3 个，数字 3 个，其他 2 个（空格和 `!`）。

2.10 综合案例

2.10.1 案例：温度转换

```c
#include <stdio.h>
int main()
{
    float celsius, fahrenheit;

    printf("请输入摄氏温度: ");
    scanf("%f", &celsius);

    // F = C * 9/5 + 32
    fahrenheit = celsius * 9.0 / 5.0 + 32;

    printf("%.1f 摄氏度 = %.1f 华氏度\n", celsius, fahrenheit);
    return 0;
}
```

2.10.2 案例：大小写互换（位运算技巧）

大写字母与小写字母的 ASCII 码相差 32（二进制第 5 位不同），用异或把这一位翻转，就能实现大小写互换。

```c
#include <stdio.h>
int main()
{
    char ch;

    printf("请输入一个字母: ");
    scanf(" %c", &ch);

    printf("原字符: %c (编码 %d)\n", ch, ch);
    printf("转换后: %c (编码 %d)\n", ch ^ 32, ch ^ 32);   // 大写?小写
    return 0;
}
```

运行示例（输入 `G`）：

```
原字符: G (编码 71)
转换后: g (编码 103)
```

> 异或的第 5 位（值为 32 的那一位）在 `'A'`(0100 0001) 与 `'a'`(0110 0001) 之间不同，翻转它就是切换大小写。

2.11 本章小结

知识点	要点
数据类型	int, float, double, char 四种基本类型
字符编码	`'0'`=48、`'A'`=65、`'a'`=97；大小写相差 32；编码是连续的
转义字符	`\n` `\t` `\\` `\'` `\"` `\0` 等；`\ddd`、`\xhh` 用编码表示字符
算术运算	`+ - * / %`，注意整数除法的截断
自增自减	`++i`（前缀）与 `i++`（后缀）的区别
赋值运算	`=` 与 `==` 区分；复合赋值右侧是一个整体
逗号运算	从左到右依次求值，整个表达式的值取最后一个；优先级最低
位运算	`& | ^ ~ << >>` 只能用于整型；`x & 1` 判奇偶，`x << n` 相当于乘 2^n
关系运算	结果为1（真）或0（假），`==` 是判断相等
逻辑运算	`&&` 与 `||` 的短路特性；与 `&`、`|` 完全不同
字符输入输出	`printf`/`scanf`、`putchar`/`getchar`；`%c` 不会跳过空白字符

知识点	要点
类型转换	隐式自动转，显式用 `(类型)`

ASCII码记忆方法

快速记忆口诀
零四八，大六五，小九七。
大小写，差三二。
数字零到九，就在四八后。
回车十三，换行十，制表就是九

2.12 课后作业

基础练习

1. 选择题

(1) 下列合法的标识符是（ ）
A. 2a　B. _count　C. int　D. a-b

(2) `5 / 2` 的值是（ ）
A. 2.5　B. 2　C. 2.0　D. 3

(3) 表达式 `3 && 0` 的值是（ ）
A. 1　B. 0　C. 3　D. true

(4) 表达式 `5 & 3` 的值是（ ）
A. 1　B. 3　C. 7　D. 15

(5) 表达式 `5 | 3` 的值是（ ）
A. 1　B. 7　C. 8　D. 15

(6) 表达式 `1 << 3` 的值是（ ）
A. 3　B. 6　C. 8　D. 16

(7) 字符 `'0'` 的 ASCII 码值是（ ）
A. 0　B. 48　C. 65　D. 97

(8) 表示空字符（ASCII 码为 0）的转义字符是（ ）
A. `'\n'`　B. `'\0'`　C. `'0'`　D. `'\\'`

(9) 逗号表达式 `(1, 2, 3)` 的值是（ ）
A. 1　B. 3　C. 6　D. 编译错误

(10) 表达式 `'B' - 'A'` 的值是（ ）
A. 1　B. 2　C. 65　D. 66

(11) 执行 `int a = 5; a *= 3 + 1;` 后，`a` 的值是（ ）
A. 16　B. 20　C. 12　D. 8

(12) 下列写法正确的是（ ）
A. `char c = "A";`　B. `char c = 'AB';`　C. `char c = 'A';`　D. `char c = A;`

2. 程序阅读

(1) 写出以下程序输出：

```
int a = 5, b = 2;
float c = a / b;
printf("%f\n", c);
```

(2) 写出以下程序输出：

```
int a = 5;
a += 3;
a *= 2;
printf("%d\n", a);
```

(3) 写出以下程序输出：

```
char ch = 'a';
printf("%c %d\n", ch ^ 32, ch ^ 32);
```

(4) 写出以下程序输出：

```
int x = 10;
printf("%d\n", x >> 1);
printf("%d\n", x & 1);
printf("%d\n", x | 1);
printf("%d\n", (x = 3, x + 7));
```

编程练习

3. 逆序输出
输入一个三位正整数（如 123），输出其逆序数（321）。

4. 时间换算
输入一个总秒数，将其转换为 "小时:分钟:秒" 格式输出。

```
输入: 3666
输出: 1:1:6
```

5. 圆的计算
输入圆的半径，计算并输出圆的周长和面积。（π取3.14）

6. 字符转换
输入一个大写字母，输出对应的小写字母。（提示：大写字母 ASCII 码 + 32 = 小写字母）

7. 判断奇偶（用位运算）
输入一个整数，用 `x & 1` 判断它是奇数还是偶数，输出"奇数"或"偶数"。（不要用 `%` 运算）

8. 位运算交换两数
输入两个整数，不使用第三个变量，用异或 `^` 交换它们的值后输出。

9. 输出二进制
输入一个 0~255 之间的整数，用位运算（`>>` 和 `&`）输出它的 8 位二进制形式。

```
输入: 5
输出: 00000101
```

10. 字符判断
输入一个字符，输出它是"大写字母""小写字母""数字字符"还是"其他字符"，并把大写转小写、小写转大写（其他字符原样输出）。要求程序对"输入数字后再输入字符"的情况也能正常工作（注意缓冲区里的回车）。

11. 字符统计
输入一行字符（以回车结束），用 `getchar()` 循环读入，统计其中字母、数字、空格和其他字符的个数并输出。

```
输入: Abc 123!
输出:
字母: 3
数字: 3
空格: 1
其他: 1
```

12. 转义字符输出
用 `printf` 和转义字符输出下面这张"名片"（注意引号、反斜杠和制表符的写法）：

```
========== 名片 ==========
姓名: 小明
班级: 2026级1班
座右铭:     "多敲代码，少走弯路"
路径: C:\Dev-Cpp\test.c
==========================
```

> **学习建议**：运算符优先级不要死记硬背，遇到不确定的加括号 `()` 即可。